

# TLM 3.x Interview Master Gap & Deep-Dive Guide

Structured / Topic-wise / Interview-Ready Edition

For 5+ year SmartStream TLM Techno-Functional Consultant interviews — Singapore banking focus

**Physical object name is implementation/version/client dependent — verify against the client schema/configuration. This PDF deliberately uses logical objects and SQL placeholders.**

Use this after the baseline PDF. It fills deep gaps and gives clean interview answers, investigation paths, SQL patterns and production support matrices without hallucinating client-specific implementation details.

# How to use this PDF

- Read section-wise. Each part is isolated so you can revise one topic quickly before interview.
- For every backend question, answer first at logical layer level, then say how you would verify the physical object in the client schema/configuration.
- For cash questions, always include account, currency, amount, sign, value date, reference, ledger/statement side, proof and exception workflow.
- For production questions, never jump directly to reload/restart. Prove file, load, item, recon, workflow, queue, matching, exception and UI layer.

## Contents

|    |                                                   |
|----|---------------------------------------------------|
| 1  | Logical database architecture                     |
| 2  | Complete DB data flow and reverse troubleshooting |
| 3  | Static creation from zero                         |
| 4  | Message feed and header controls                  |
| 5  | Cash reconciliation master                        |
| 6  | Mirror vs double entry                            |
| 7  | Matching deep dive                                |
| 8  | Proposal triggering                               |
| 9  | Workflow queue / MMQ                              |
| 10 | SQL interview patterns                            |
| 11 | MI / DMW / Informatica                            |
| 12 | Tectia / Linux / ServiceMix                       |
| 13 | Smart Studio / TLM View / WebConnect              |
| 14 | Position reconciliation                           |
| 15 | Exception management                              |
| 16 | Production incidents                              |
| 17 | Flagship Q&A                                      |
| 18 | Final survival appendices                         |
| 19 | 7-day learning plan                               |

# Part 1 — Logical Database Architecture

**Interview rule: when asked 'which table?', never guess. Say the physical object is implementation/version/client dependent, then identify the logical area and the evidence you would query.**

The table below is intentionally logical. It is the safe structure you can explain in any TLM 3.x interview and then map to the client's verified schema.

| Logical area         | Physical objects                                      | When requested                       | Key identifiers                      | How to query                       | Failure looks like                                 |
|----------------------|-------------------------------------------------------|--------------------------------------|--------------------------------------|------------------------------------|----------------------------------------------------|
| Static/config        | Set, recon, class, feeds, accounts, rules, workflow   | Created/changed during configuration | codes, status, effective date, owner | query active config by code/date   | wrong/inactive config routes data incorrectly      |
| Set                  | Operational container for related recon setup         | During static build                  | set_code, set_id, BU                 | find linked recons/accounts        | records not visible or wrong operational grouping  |
| Reconciliation       | Concrete recon process                                | During static build                  | recon_id, recon_code, class          | query recon by code/status         | items load but are not eligible for expected recon |
| Reconciliation Class | Model/type: cash/position/security/intersystem        | Model/static setup                   | class_code, class_type               | verify recon class/dimensions      | wrong fields/proofs/rules available                |
| Data Source          | Logical recon side: Ledger, Statement, Custodian      | Recon setup and item association     | data_source, side                    | count items by source/side         | both sides mapped to same side or one side missing |
| Message Feed         | Inbound stream definition                             | Feed config and runtime load         | feed_code, source, format, mapping   | query loads/items by feed          | wrong feed, wrong side, wrong mapping              |
| Account              | Business control account / nostro / depot / GL        | Static/reference setup               | account_id, alias, currency, BU      | query account alias/effective date | lookup reject or wrong account break               |
| Source/reference     | Source systems, aliases, currency, asset, customer    | Static sync/admin update             | source_code, alias, effective date   | lookup by source value             | lookup miss or stale alias                         |
| Message Type         | CSV, MT940, MT950, camt, MT535, proprietary           | Feed setup                           | message_type, version                | query feed by message type         | parser/validation mismatch                         |
| File metadata        | file name, size, date, checksum, counts               | File pickup/registration             | file_id, file_name, business_date    | query by file/feed/date            | file present but ignored/duplicate/partial         |
| Message header       | statement/message envelope: sender, account, sequence | Parsing                              | message_id, sequence, account        | query by message ref/account       | records load to wrong account/date                 |
| Load/batch control   | run status and counts                                 | Loader start/end                     | load_id, status, accepted, rejected  | query latest load by file          | failed/partial/duplicate load                      |
| Load detail          | record-level trace                                    | Per record                           | load_id, line_no, source_ref         | query by line/ref                  | specific record missing or wrong target            |
| Reject/error         | validation/parsing/DB errors                          | When record/file fails               | load_id, error_code, field           | query rejects by load/error        | partial or full batch failure                      |
| BDR/item             | canonical reconcilable item                           | After accepted mapping               | item_id, load_id, account, ref       | query by ref/file/account          | no item or wrong amount/date/ref                   |
| Recon status         | item linked to recon lifecycle                        | After item/static association        | item_id, recon_id, status            | query item recon/source/status     | loaded but not picked by matching                  |
| Matching/pass        | pass diagnostics and quality                          | During matching                      | run_id, pass_id, result              | query pass result by item/run      | no proposal or failed quality                      |
| Proposal             | candidate match group                                 | After pass quality                   | proposal_id, status, quality         | query proposal by item             | pending/not approved/not visible                   |
| Match history        | final match/audit history                             | Auto/manual match                    | match_id, item_id, timestamp         | query item-to-match                | false/missing/duplicate match                      |
| Workflow             | process state and transitions                         | Initiation and user/system actions   | workflow_id, state                   | query state by item/exception      | stuck or wrong lifecycle                           |
| Initiation           | starts workflow/matching lifecycle                    | After load/schedule/manual trigger   | initiation_id, trigger_type          | query initiation status            | loaded data idle                                   |
| MMQ/workflow queue   | queued work waiting for consumer                      | After initiation/trigger             | queue_id, status, retry, age         | query queue status/age             | backlog/stuck/poison record                        |
| Queue history        | attempts and errors                                   | Every queue processing attempt       | queue_id, attempt_no, error          | query attempts by queue id         | same error repeats                                 |

| Logical area       | Purpose / status               | When populated                 | Key identifiers                    | How to query                | Failure looks like               |
|--------------------|--------------------------------|--------------------------------|------------------------------------|-----------------------------|----------------------------------|
| Exception/break    | actionable unresolved issue    | Failed/no match/proof/manual   | exception_id, status, owner        | query open exception by ref | break not raised or wrong owner  |
| Balance/proof      | opening/closing/movement proof | Balance load/proof calculation | account, currency/asset, date      | query proof difference      | unbalanced despite matched items |
| Audit/history      | who/what/when evidence         | Every controlled action        | object_id, action, user, timestamp | query by object/ref/user    | cannot prove change/action       |
| Security/user/role | access control                 | User provisioning/admin        | user, role, group, inbox           | query user role/inbox       | DB has data but user cannot see  |
| TLM View metadata  | dashboards, filters, columns   | UI/dashboard config            | view_id, widget, filter, role      | compare UI filter to SQL    | UI count differs from DB         |

## Dependency diagram

### Static/reference foundation

Set -> Reconciliation Class -> Reconciliation  
 -> Data Source(s) -> Message Feed(s) -> Account eligibility  
 -> Workflow + Initiation + Proposal Trigger  
 -> Scope + Population + Match Rule + Pass Quality

### Runtime lineage

Source -> File/Message -> File Metadata -> Message Header  
 -> Load/Batch -> Load Detail -> Reject OR BDR Item  
 -> Recon Status -> Workflow Initiation -> Queue/MMQ  
 -> Scope -> Population -> Pass Quality -> Proposal  
 -> Match OR Exception -> Audit -> TLM View

## Part 2 — Complete DB Data Flow

Use this table to answer 'what happens next?' and 'how do you prove it?' follow-ups.

| Step                  | Context in                     | Logical storage/change           | Next trigger             | Proof                                |
|-----------------------|--------------------------------|----------------------------------|--------------------------|--------------------------------------|
| Source system         | extract/message/control totals | no TLM write yet                 | source scheduler/job     | source log/control total             |
| File/message          | raw payload                    | file metadata after pickup       | transfer or middleware   | size, header/trailer, grep reference |
| Tectia/SFTP           | secure transfer                | landing file/transfer log        | complete transfer/marker | timestamp, checksum, permissions     |
| Landing directory     | file ready                     | filesystem state                 | poller/route/loader      | ls/find/df/permissions               |
| ServiceMix/middleware | route/event                    | integration log/queue            | route invokes loader     | route status, endpoint, logs         |
| DMW/MI/ETL            | file/message records           | load/reject/map logs             | parser/mapper            | accepted+rejected, reject reasons    |
| Header                | business date/account/count    | file/message header logical area | validation success       | header vs expected date/account      |
| Load/batch            | loader run                     | load id/status/counts            | commit/next stage        | status, counts, timings              |
| BDR/item              | accepted record                | canonical item                   | static association       | item by ref/account/date             |
| Static association    | item + static keys             | recon/status/source/account      | initiation/scope         | item recon/data source/status        |
| Workflow initiation   | eligible item/recon            | workflow/initiation state        | queue create             | initiation timestamp/status          |
| Queue/MMQ             | work object                    | queue status/retry/age           | consumer/service         | backlog, oldest, retry               |
| Scope                 | open eligible items            | candidate set                    | matching run             | candidate count                      |
| Population            | candidate groups               | group diagnostics                | match pass               | counterpart in same group?           |
| Pass quality          | comparison result              | pass result/quality              | proposal/match/break     | why pass/fail                        |
| Proposal              | candidate match                | proposal id/status               | auto/manual action       | proposal exists/visible              |
| Match                 | accepted group                 | match history/status             | exception closure/audit  | item status matched                  |
| Exception             | break                          | exception/workflow/audit         | owner investigation      | open exception visible               |
| TLM View              | dashboard request              | view/filter/security             | user action/report       | UI count vs DB criteria              |

### Reverse troubleshooting path

- 1 TLM View missing: capture dashboard, filters, role, business date and exact search criteria.
- 2 DB item count: reproduce UI filters in SQL. If DB has rows, investigate security/view filters.
- 3 Exception: check open/closed/cancelled exception by reference/account/date.
- 4 Match: if already matched, no open break may be expected; if false match, inspect pass/audit.
- 5 Proposal: check pending/suggested/perfect proposals and approval status.
- 6 Queue: check old, retrying or error queue entries.
- 7 Workflow: confirm initiation and current state.
- 8 Item: confirm canonical item exists with expected account/source/recon/date.
- 9 Load: confirm successful batch and accepted/rejected counts.
- 10 Feed: confirm correct feed and data source side.
- 11 File: confirm arrival, size, count, checksum, naming.
- 12 Source: if record absent, escalate upstream with evidence.

## Part 3 — Static Creation from Zero

**Static creation changes configuration/reference/audit logical areas. It should not create transaction items, proposals, matches or exceptions until runtime data is loaded and processed.**

| Object           | Mandatory logical fields                                    | Dependencies                     | Expected result         | Common error                | DB logical impact / test      |
|------------------|-------------------------------------------------------------|----------------------------------|-------------------------|-----------------------------|-------------------------------|
| Set              | unique code/name, BU, active/effective date                 | BU/security/naming               | set active and linked   | duplicate/inactive set      | query static metadata + audit |
| Reconciliation   | code/name, set, class, sources, workflow, trigger           | set/class/accounts               | recon eligible          | wrong class/inactive        | query recon config            |
| Data sources     | side codes Ledger/Statement                                 | recon/class                      | separate sides          | both feeds same side        | count by data source          |
| Message feeds    | feed code, message type, file pattern, mapping, target side | source/data source/static        | file picked correctly   | wrong feed/mapping          | query load by feed            |
| Accounts         | account id, alias, currency, BU, active dates, recon link   | currency/source/BU               | items map correctly     | lookup reject/wrong account | query account alias           |
| Workflow         | states, queues, roles, SLA, exception types                 | recon/security                   | items/exceptions route  | stuck/invisible             | query workflow state          |
| Initiation       | event, conditions, schedule/manual, target queue            | workflow/recon/feed availability | queue created           | loaded data idle            | query initiation/queue        |
| Scope            | recon/status/date/account/source eligibility                | item fields                      | expected candidates     | no candidates               | candidate count               |
| Population       | grouping keys                                               | scope/match fields               | right counterpart group | bad group/explosion         | diagnostics/proposal          |
| Match rule       | amount/date/ref/currency/account/sign/tolerance             | population fields                | expected pass/fail      | false match                 | pass result                   |
| Pass quality     | perfect/suggested/pending thresholds                        | match rule results               | correct quality         | weak auto-match             | quality result                |
| Proposal trigger | any/all/always/never/scheduled/manual                       | feeds/workflow/service           | right proposal timing   | premature/missing           | trigger + queue + run         |

### Static vs transaction interview answer

When static is created, I expect configuration/reference/audit areas to change. When a file loads, load metadata, rejects and item/business data areas change. When matching runs, pass/proposal/match/status areas change. When a break is created, exception/workflow/audit areas change. I do not name a physical table unless I verify the client schema.

## Part 4 — Message Feed / Header Control

| Control                | Meaning                                                        | Failure                          | Evidence                       |
|------------------------|----------------------------------------------------------------|----------------------------------|--------------------------------|
| Feed code / short code | identifies inbound stream; short code may appear in names/logs | wrong feed selected              | feed setup + load metadata     |
| Message type           | CSV/MT940/MT950/camt/MT535/proprietary                         | parser mismatch                  | feed config + parser log       |
| Filename               | source/date/sequence/extension convention                      | not picked or wrong date         | ls + load metadata             |
| Header                 | date/account/statement sequence/sender                         | wrong account/date               | header logical area/parser log |
| Trailer                | record count/control total                                     | partial or count mismatch        | trailer vs accepted+rejected   |
| Business date          | recon eligibility date                                         | loaded but not in today's view   | load/item business date        |
| Feed-to-data-source    | maps feed to Ledger/Statement side                             | load succeeds but matching fails | items by feed/source           |
| Duplicate control      | filename/hash/source batch/sequence/business key               | duplicate items/matches          | duplicate loads/items          |
| Malformed/zero/partial | parser or file integrity failure                               | full/partial reject              | size, checksum, rejects        |
| Wrong mapping          | field values wrong after load                                  | false breaks                     | source-to-item comparison      |

A file can load successfully but fail reconciliation association if account alias is missing/inactive, business date is outside scope, feed is mapped to wrong data source, recon/account is inactive, required static is missing, or initiation/trigger conditions are not met.

## Part 5 — Cash Reconciliation Master

| Cash topic                     | Interview-ready explanation                                                                |
|--------------------------------|--------------------------------------------------------------------------------------------|
| Nostro                         | our account with another bank; internal ledger vs external statement                       |
| Vostro                         | their account with us; relevant in correspondent banking but different control perspective |
| Ledger/Internal GL             | internal postings; may have debit/credit legs and sign conventions                         |
| Statement/Bank statement       | external bank view of movements and balances                                               |
| MT940/MT950/camt               | cash statement/reporting formats; parser mapping is client dependent                       |
| Opening + Movement = Closing   | proof equation; transaction matching alone is not enough                                   |
| Value/posting/transaction date | different dates; value date often key for cash matching                                    |
| Amount/currency/sign           | normalize debit/credit perspective before comparing                                        |
| Reference/narrative            | matching identifier; may need trimming, masking, narrative extraction                      |
| Bank charges/FX/tolerance      | business-approved differences; do not hide with broad tolerance                            |
| Reversal/timing/duplicate      | common break causes requiring source lineage and audit                                     |

### Cash proof example

External opening 100,000 + credits 25,000 - debits 15,000 = external closing 110,000. If internal ledger movements match transaction-to-transaction but internal opening is 99,500, the account is still unbalanced. A senior consultant checks both transaction matching and balance proof.

## Part 6 — Mirror vs Double Entry

**Do not state mirror/double-entry behavior is identical in every TLM environment. Explain accounting/reconciliation concept and verify the actual client implementation.**

| Scenario                                      | Reconciler explanation                                                                                                                                      |
|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Statement CREDIT 10,000 / Ledger DEBIT 10,000 | Incoming cash may be credit on statement and debit increase internally. Normalize to same economic +10,000 before matching.                                 |
| Statement DEBIT 10,000 / Ledger CREDIT 10,000 | Outgoing cash may be debit externally and credit decrease internally. Normalize direction/sign.                                                             |
| Ledger has DR + CR legs                       | Double-entry means accounting completeness. Reconciliation usually selects the cash/nostro leg or approved derived item; do not match both GL legs blindly. |
| Multiple ledger postings to one statement     | Use one-to-many/netting with strong population and pass quality; preserve lineage.                                                                          |
| Partial double-entry                          | One accounting leg missing/timing delayed. Do not force a mirror match; investigate source/accounting balance.                                              |
| Wrong sign                                    | Creates false amount mismatch or false match; check debit/credit mapping and normalized amount.                                                             |
| Duplicate posting                             | One statement should not incorrectly match duplicate ledger postings unless aggregate rule intentionally expects them.                                      |
| Timing difference                             | Ledger side may arrive before statement; trigger/ageing design controls false break noise.                                                                  |

## Part 7 — Matching Deep Dive

| Stage        | Purpose                    | Cash ABC123 walk-through                              |
|--------------|----------------------------|-------------------------------------------------------|
| Scope        | select eligible open items | open USD account A items for 20-Aug                   |
| Population   | group possible candidates  | ledger and statement ref ABC123 grouped               |
| Match rule   | compare fields             | amount 1000, currency USD, ref ABC123, date 20-Aug    |
| Pass quality | classify quality           | perfect if all mandatory exact; suggested if tolerant |
| Proposal     | candidate match group      | proposal links ledger and statement item              |
| Match        | final accepted state       | auto/manual match updates status                      |
| Exception    | unresolved issue           | mismatch/missing side routed to owner                 |

### Mismatch matrix

| Mismatch           | Diagnostic focus                                             |
|--------------------|--------------------------------------------------------------|
| amount mismatch    | check sign, charges, FX, decimal conversion, partial payment |
| date mismatch      | check value vs posting date and tolerance window             |
| reference mismatch | check normalization, masks, narrative extraction             |
| currency mismatch  | check mapping/defaults and cross-currency policy             |
| missing ledger     | trace ledger feed/load/reject/static                         |
| missing statement  | check statement arrival/timing/trigger                       |
| duplicate          | check duplicate file/business key and source duplicates      |
| tolerance          | check limit, approval, auto/suggested behavior               |
| bank charge        | check fee policy/write-off/exception route                   |
| timing difference  | monitor ageing; do not force match                           |

## Part 8 — Proposal Triggering

| Trigger               | Interview explanation                                                                                        |
|-----------------------|--------------------------------------------------------------------------------------------------------------|
| Any Matching Input    | run/propose when any configured side arrives; useful for real-time visibility but can create one-sided noise |
| All Required Inputs   | run when required sides exist; useful when statement/balance must be present                                 |
| Always Propose        | generate proposals when eligible data exists; exact behavior/client term varies                              |
| Never Propose         | no automatic proposal; manual/scheduled action may be required                                               |
| Scheduled/date-driven | EOD/month-end/cutoff based trigger where supported                                                           |
| Feed availability     | checks whether required feed/balance arrived                                                                 |
| Initiation            | workflow state/queue must exist                                                                              |
| Matching service      | service must consume queue and create results                                                                |

### Daily ledger / monthly statement

- 1 Daily ledger arrives: file loads, items are created, static/recon association occurs.
- 2 If trigger requires all selected inputs or month-end schedule, final proposals should not run yet.
- 3 Items may be open/pending or monitored as missing statement depending design.
- 4 On 31st, statement arrives and feed availability condition is satisfied.
- 5 Workflow initiation creates queue entry; matching service consumes it.
- 6 Scope/population/pass quality generate proposals, matches or exceptions.
- 7 Balance proof validates opening + movements = closing.

## Part 9 — Workflow Queue / MMQ

**MMQ and workflow\_queue names/implementation are client dependent. Answer by logical queue concept and verify process names from runbook/logs.**

| Topic                     | Answer                                                                                                                        |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Who polls workflow_queue? | A configured workflow/matching/integration processor or scheduler. Exact service name is client dependent; verify in runbook. |
| Queue entry               | object id, type, status, priority, created time, retry count, error                                                           |
| Retry                     | failed work retried under policy; repeated same error suggests poison record                                                  |
| Poison record             | bad record/config/data repeatedly failing and potentially blocking progress                                                   |
| Backlog/ageing            | count by status and oldest created time shows stuck processing                                                                |
| Service down              | loads succeed and queue grows, but no proposals/matches appear                                                                |
| Prove matching started    | queue consumed, matching run/pass results logged, proposal/match/exception timestamps exist                                   |

## Part 10 — SQL Interview Patterns

Use placeholders until the client schema is verified. Status values and joins must be confirmed from SmartSchema, data dictionary, runbook or known support SQL.

| Question                     | General SQL pattern                                                                                                                                                      |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| file-wise matched count      | SELECT b.<FILE_NAME>, COUNT(*) FROM <TLM_ITEM_TABLE> i JOIN <TLM_LOAD_BATCH_TABLE> b ON b.<LOAD_ID>=i.<LOAD_ID> WHERE i.<MATCH_STATUS>='MATCHED' GROUP BY b.<FILE_NAME>; |
| unmatched count              | filter NVL(match_status,'UNMATCHED') <> 'MATCHED' and group by file/account/recon                                                                                        |
| match percentage             | 100 * matched count / total count using CASE                                                                                                                             |
| break percentage             | 100 * non-matched/open exception count / total                                                                                                                           |
| duplicate references         | GROUP BY reference HAVING COUNT(*) > 1                                                                                                                                   |
| duplicate files              | GROUP BY file_name, business_date/source HAVING COUNT(*) > 1                                                                                                             |
| latest load                  | ORDER BY load_end_ts DESC FETCH FIRST / ROWNUM=1                                                                                                                         |
| failed/partial load          | status failed OR expected_count <> accepted+rejected                                                                                                                     |
| reject count                 | GROUP BY error_code/field for load_id                                                                                                                                    |
| items by feed/account/recon  | GROUP BY feed_id/account_id/recon_id, status                                                                                                                             |
| proposals/matches/exceptions | query logical proposal/match/exception area by item/recon/status                                                                                                         |
| queue ageing                 | GROUP BY queue_status; MIN(created_ts) oldest                                                                                                                            |
| workflow status              | GROUP BY workflow_state                                                                                                                                                  |
| balance difference           | internal balance - external balance by account/currency/asset/date                                                                                                       |
| opening + movement = closing | opening + SUM(movements) - closing as proof_diff                                                                                                                         |
| source-to-item lineage       | file/load -> item by source reference                                                                                                                                    |
| item-to-match lineage        | item -> match history                                                                                                                                                    |
| item-to-exception lineage    | item -> exception status/owner                                                                                                                                           |

## Part 11 — MI / DMW / Informatica

**Do not invent MI syntax. Explain conceptual source-target mapping, transformations, lookups, validations and error action.**

| MI example                 | Conceptual mapping                                                                                                                                               |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CSV cash ledger            | account_no -> account lookup; ccy -> currency; dr_cr + amount -> normalized amount; value_date parse; reference trim/uppercase; reject lookup/date/amount errors |
| MT940 statement            | parse statement account, balances and transaction lines; normalize debit/credit; extract reference/narrative; validate sequence/count/balance where configured   |
| Double-entry normalization | select cash/nostro leg or approved derived item; normalize sign; validate balanced posting if required; never blindly match both GL legs                         |

### DMW vs MI vs Informatica

| Task        | Strong formal answer                                                                                                                                          |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DMW         | Hands-on source-to-target mapping, transformations, validations, rejects and count reconciliation.                                                            |
| MI          | TLM message/file loading concept; exact syntax and implementation client dependent.                                                                           |
| Informatica | Understand mapping/session/workflow/lookup/expression/filter/router/joiner/aggregator/update strategy/recovery; do not fabricate deep experience if not used. |

## Part 12 — Tectia / Linux / ServiceMix

| Issue             | Command / check                                                                               |
|-------------------|-----------------------------------------------------------------------------------------------|
| latest file       | ls -ltr &lt;dir>   tail; robust: find &lt;dir> -type f -printf '%T@ %p\n'   sort -n   tail -1 |
| permissions       | ls -l, stat, id &lt;loader_user>;                                                             |
| size/count        | ls -lh, stat -c%s, wc -l                                                                      |
| grep reference    | grep -n 'ABC123' &lt;file>;                                                                   |
| disk/process/logs | df -h, du -sh, ps -ef   grep, tail -100, grep -i error                                        |

### ServiceMix logical checks

- Service/bundle active
- Endpoint reachable
- Route enabled and moving file/message
- Datasource/DB connectivity
- Logging errors
- Queue backlog
- Environment properties changed
- Never invent exact cfg filenames; verify deployment runbook.

## Part 13 — Smart Studio / TLM View / WebConnect

| Topic                      | Answer                                                                                     |
|----------------------------|--------------------------------------------------------------------------------------------|
| Smart Studio / Recon Admin | solution/static/model configuration depending version/client                               |
| TLM Control                | workflow/BPM lifecycle, queues, approvals, routing                                         |
| TLM View                   | browser dashboards and operational UI with filters/security                                |
| WebConnect                 | web access/integration term; exact role client dependent                                   |
| SmartSchema                | metadata layer over database/business structures                                           |
| Create dashboard           | define users, dataset, columns, filters, sorting, grouping, roles, test as user            |
| Add/change column          | expose existing attribute; header is UI label, not DB column unless model changes          |
| Missing records            | check UI filters, role/security, recon status, workflow, backend count under same criteria |

## Part 14 — Position Reconciliation and Balance

Position recon is secondary here but must be interview-ready. It compares internal holdings/positions with custodian/depot positions by account/depot, security/ISIN, quantity, trade date, settlement date and settled/unsettled bucket. The logical position balance area stores opening position, movements and closing position by account/security/date/bucket. The physical table name must be verified in the client schema.

A position can break even when transaction movements match if opening balance, corporate action, settlement bucket, transfer, security alias or custodian timing differs.

## Part 15 — Exception / Break Management

MATCH -> NO MATCH -> BREAK -> EXCEPTION -> QUEUE -> OWNER -> INVESTIGATION -> COMMENT/EVIDENCE -> RESOLUTION -> APPROVAL IF REQUIRED -> CLOSE -> AUDIT.

- Include ageing, SLA, owner/inbox, escalation, reason code and resolution code.
- Subsequent matching may automatically clear/close exceptions depending configuration.
- Every manual match, comment, reassignment and closure should have audit evidence.

## Part 16 — Production Incident Matrix

| Incident               | Symptom                                                                               | Evidence                                 | Fix + communication                                                          |
|------------------------|---------------------------------------------------------------------------------------|------------------------------------------|------------------------------------------------------------------------------|
| file missing           | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| file late              | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| zero byte              | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| duplicate              | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| partial load           | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| mapping failure        | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| lookup failure         | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| wrong account          | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| wrong data source      | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| wrong feed             | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| matching not triggered | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| queue stuck            | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| service down           | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| proposal not generated | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| false match            | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| low match rate         | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| exception spike        | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| view mismatch          | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| balance mismatch       | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |

| Incident          | Checklist                                                                             | Evidence                                 | Fix / Communication                                                          |
|-------------------|---------------------------------------------------------------------------------------|------------------------------------------|------------------------------------------------------------------------------|
| timing difference | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| duplicate item    | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| wrong sign        | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| wrong currency    | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| wrong date        | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |
| wrong reference   | symptom -> check source/file/feed/load/item/recon/workflow/queue/match/exception/view | DB logical layer + logs + service status | fix under runbook; validate counts/status; communicate impact/ETA/prevention |

## Part 17 — Flagship Interview Answers

### Tell me about yourself

**20-second answer:** I am a TLM techno-functional consultant focused on reconciliations, SQL, data mapping and production support.

**Senior answer:** I am a TLM techno-functional consultant focused on reconciliations, SQL, data mapping and production support. I can trace a transaction from source file to TLM item, workflow, match/exception and TLM View. My strongest hands-on areas are SQL and DMW/data mapping; I am honest where tools such as Informatica or MI syntax are client-specific.

**Trap:** Do not invent physical names or overclaim client-specific behavior.

### Create new reconciliation

**20-second answer:** Requirement -> sources -> static -> mapping -> load validation -> scope/population/pass quality -> proposal trigger -> workflow/exceptions -> TLM View -> unit/SIT/UAT -> deployment -> monitoring.

**Senior answer:** Requirement -> sources -> static -> mapping -> load validation -> scope/population/pass quality -> proposal trigger -> workflow/exceptions -> TLM View -> unit/SIT/UAT -> deployment -> monitoring.

**Trap:** Do not invent physical names or overclaim client-specific behavior.

### Mandatory static fields

**20-second answer:** Set/recon code, class, active/effective status, data sources, feeds, accounts, workflow, initiation, rules, trigger, ownership/security and audit.

**Senior answer:** Set/recon code, class, active/effective status, data sources, feeds, accounts, workflow, initiation, rules, trigger, ownership/security and audit. Exact screen fields vary.

**Trap:** Do not invent physical names or overclaim client-specific behavior.

### Recon Class vs Data Source

**20-second answer:** Class is model/type; data source is side/input role.

**Senior answer:** Class is model/type; data source is side/input role. Cash class with Ledger and Statement sources is the clean example.

**Trap:** Do not invent physical names or overclaim client-specific behavior.

### MI syntax

**20-second answer:** Do not invent.

**Senior answer:** Do not invent. Explain conceptual source definition, record layout, mapping, transformation, lookup, validation, target association and error action.

**Trap:** Do not invent physical names or overclaim client-specific behavior.

## Proposal triggering

**20-second answer:** Separate load, initiation, trigger and matching.

**Senior answer:** Separate load, initiation, trigger and matching. Daily ledger/monthly statement usually needs all-required or scheduled trigger if final proposals wait for statement.

**Trap:** Do not invent physical names or overclaim client-specific behavior.

## Who polls workflow\_queue

**20-second answer:** A configured queue consumer such as workflow/matching processor; exact service and table names are client dependent.

**Senior answer:** A configured queue consumer such as workflow/matching processor; exact service and table names are client dependent.

**Trap:** Do not invent physical names or overclaim client-specific behavior.

## Backend tables

**20-second answer:** Answer by logical areas: static, feed/load, reject, item, recon status, workflow/queue, matching/proposal/match, exception, balance, audit, security, view metadata.

**Senior answer:** Answer by logical areas: static, feed/load, reject, item, recon status, workflow/queue, matching/proposal/match, exception, balance, audit, security, view metadata.

**Trap:** Do not invent physical names or overclaim client-specific behavior.

## TLM View dashboard

**20-second answer:** Define dataset/columns/filters/sorting/grouping/security; test as user; compare backend counts with same filters.

**Senior answer:** Define dataset/columns/filters/sorting/grouping/security; test as user; compare backend counts with same filters.

**Trap:** Do not invent physical names or overclaim client-specific behavior.

## ServiceMix cfg files

**20-second answer:** Do not name files.

**Senior answer:** Do not name files. Discuss service/bundle, endpoint, route, datasource, logging, queue and environment properties.

**Trap:** Do not invent physical names or overclaim client-specific behavior.

# Final Survival Appendices

## 100 must-remember concepts

| #  | Concept                                 | #  | Concept                                     |
|----|-----------------------------------------|----|---------------------------------------------|
| 1  | Physical names are client dependent     | 51 | Wrong source kills matching                 |
| 2  | Logical layer first                     | 52 | Wrong class gives wrong fields              |
| 3  | Static is not transaction data          | 53 | Any/all/scheduled/manual triggers           |
| 4  | Load is not matching                    | 54 | Daily ledger/monthly statement needs design |
| 5  | Trigger is not matching                 | 55 | Queue ageing shows stuck processing         |
| 6  | Initiation creates workflow/queue       | 56 | Poison records repeat failures              |
| 7  | Queue consumer creates next state       | 57 | Service down causes backlog                 |
| 8  | Matching service must run               | 58 | DB locks can block queue                    |
| 9  | Scope selects eligible items            | 59 | Logs complement SQL                         |
| 10 | Population groups candidates            | 60 | Linux proves file state                     |
| 11 | Match rule compares                     | 61 | Tectia is secure transfer                   |
| 12 | Pass quality categorizes                | 62 | ServiceMix is integration                   |
| 13 | Proposal is candidate group             | 63 | Do not invent filenames                     |
| 14 | Match is final accepted state           | 64 | MI syntax is client dependent               |
| 15 | Exception is actionable break           | 65 | MI maps/transforms/validates                |
| 16 | Audit proves actions                    | 66 | DMW maps source to target                   |
| 17 | TLM View is filtered/permissioned       | 67 | Informatica concepts transfer               |
| 18 | SmartSchema is metadata                 | 68 | Be honest about exposure                    |
| 19 | Recon class is model                    | 69 | Unit tests prove mapping                    |
| 20 | Data source is side                     | 70 | SIT proves integration                      |
| 21 | Message feed is inbound stream          | 71 | UAT proves business acceptance              |
| 22 | Message type is format                  | 72 | Regression protects rules                   |
| 23 | File metadata proves arrival            | 73 | Volume testing matters                      |
| 24 | Header can drive account/date           | 74 | Config management matters                   |
| 25 | Load control proves run                 | 75 | Maker-checker matters                       |
| 26 | Rejects prove validation failure        | 76 | Role access matters                         |
| 27 | BDR item is canonical                   | 77 | Dashboard labels are not DB columns         |
| 28 | Recon association makes eligible        | 78 | UI missing may be security/filter           |
| 29 | Balance proof can break despite matches | 79 | Backend count must match UI filters         |
| 30 | Nostro is our account at another bank   | 80 | Control totals matter                       |
| 31 | Ledger is internal                      | 81 | Source lineage matters                      |
| 32 | Statement is external                   | 82 | Load id matters                             |
| 33 | Debit/credit perspective differs        | 83 | File id matters                             |
| 34 | Sign normalization is critical          | 84 | Item id matters                             |
| 35 | Mirror is not double-entry              | 85 | Match id matters                            |
| 36 | Double-entry is accounting principle    | 86 | Exception id matters                        |
| 37 | Do not match both GL legs blindly       | 87 | Business date matters                       |
| 38 | One-to-many handles aggregation         | 88 | Value date differs from posting date        |
| 39 | Tolerance needs governance              | 89 | Header date may differ from filename        |

| #  | Concept                           | #   | Concept                            |
|----|-----------------------------------|-----|------------------------------------|
| 40 | Manual match needs audit          | 90  | Accepted+rejected should reconcile |
| 41 | Write-off needs approval          | 91  | Rerun can duplicate                |
| 42 | Bank charges need policy          | 92  | Archive carefully                  |
| 43 | FX needs agreed conversion        | 93  | Communicate impact not guesses     |
| 44 | Timing differences age            | 94  | Root cause before fix              |
| 45 | Duplicate file needs idempotency  | 95  | Validate after fix                 |
| 46 | Partial load needs count controls | 96  | Document prevention                |
| 47 | Zero byte should alert            | 97  | SQL placeholders are acceptable    |
| 48 | Wrong filename can bypass feed    | 98  | Data dictionary helps discovery    |
| 49 | Wrong feed can load wrong side    | 99  | Runbook is source of truth         |
| 50 | Wrong account causes wrong route  | 100 | Do not direct-update prod tables   |

## Final compact lists

| List                             | Items                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 50 backend investigation points  | source record, file arrival, file completeness, feed, message type, header, trailer, load batch, load detail, rejects, mapping version, lookup, item, account, data source, recon, status, workflow, initiation, queue, queue age, retry, service, locks, scope, population, pass, proposal, match, exception, balance, audit, security, dashboard, filters, role, source count, target count, duplicate file, duplicate item, sign, currency, date, reference, amount, business date, SLA, owner, communication, prevention                                                                 |
| 30 SQL patterns                  | file-wise matched count, unmatched count, match percentage, break percentage, duplicate references, duplicate files, latest load, failed/partial load, reject count, items by feed/account/recon, proposals/matches/exceptions, queue ageing, workflow status, balance difference, opening + movement = closing, source-to-item lineage, item-to-match lineage, item-to-exception lineage, count by account, count by source, latest transaction, aged exception, manual match audit, duplicate business key, null mandatory, amount totals, source-target compare, queue retries, user role |
| 25 Linux commands                | ls, ls -ltr, find, grep, tail, head, cat, less, wc, du, df, ps, top, chmod, chown, mv, cp, rm, mkdir, touch, stat, sort, uniq, awk, sed                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 25 cash scenarios                | perfect match, amount/date/reference/currency mismatch, missing ledger, missing statement, duplicate, bank charge, FX, reversal, timing difference, wrong sign, wrong account, wrong value date, opening/closing balance mismatch, manual match, write-off, partial file, reject, stale break, low match rate, false match, narrative reference, debit/credit issue, tolerance                                                                                                                                                                                                               |
| 15 mirror/double-entry scenarios | statement credit vs ledger debit, statement debit vs ledger credit, DR+CR legs, one-to-many GL postings, partial double-entry, unbalanced posting, wrong cash leg, both GL legs loaded, mirror duplicate, wrong sign, same reference DR/CR, fee leg, reversal, timing gap, manual match hides issue                                                                                                                                                                                                                                                                                          |
| 15 proposal trigger scenarios    | any input, all required, scheduled, month-end, manual rerun, always, never, missing side, late cutoff, inactive feed, balance required, initiation fail, queue not consumed, service down, trigger change                                                                                                                                                                                                                                                                                                                                                                                    |
| 20 MI/DMW scenarios              | CSV ledger, MT940, double-entry normalization, date fail, amount fail, account lookup miss, invalid currency, null reference, optional default, mandatory reject, partial load, full reject, duplicate file, duplicate record, wrong feed, count mismatch, target mismatch, promotion issue, rollback, SIT regression                                                                                                                                                                                                                                                                        |
| 20 TLM View scenarios            | create dashboard, add column, header label, filters, sorting, grouping, role access, missing records, backend count differs, saved user filter, business date, status excluded, workfolder, inbox, audit column, match column, reason/resolution, performance, export, shared view                                                                                                                                                                                                                                                                                                           |

# Final 1-page Survival Sheet

If asked 'which table?', answer: Physical object name is implementation/version/client dependent — verify against the client schema/configuration. Then identify the logical layer: static, feed/load, reject, item/BDR, recon status, workflow/queue, proposal/match, exception, balance, audit, security or view metadata.

If asked 'what happens next?', answer: load -> item -> static/recon association -> workflow initiation -> queue -> scope -> population -> pass quality -> proposal -> match/exception -> audit -> TLM View.

If asked 'how do you prove it?', use file checks, load counts, rejects, item query, recon/source/account status, workflow/queue status, match/proposal/exception result, audit and UI filter/security comparison.

If tested for bluffing, never invent TLM table names, MI syntax, ServiceMix cfg filenames, MMQ implementation, commands or direct production updates.

For cash recon, always mention account, currency, value date, amount, debit/credit sign, reference, ledger vs statement, opening balance, movements, closing balance, proof, tolerance, manual match, exception and audit.

## 7-day learning plan

- 1 Day 1: Logical DB architecture and safe 'which table' answers.
- 2 Day 2: End-to-end data flow and reverse troubleshooting.
- 3 Day 3: Cash recon, nostro, sign and balance proof.
- 4 Day 4: Matching, scope, population, pass quality and proposals.
- 5 Day 5: Workflow/MMQ/proposal triggering scenarios.
- 6 Day 6: SQL/Linux/MI/DMW drills.
- 7 Day 7: Mock interview aloud with strict no-hallucination discipline.